

Informe técnico

Sistema de gestión de inventario del Mercado Municipal de Quevedo

Loor Marlon
Universidad Técnica Estatal de Quevedo (UTEQ)
Aplicaciones Web

24 de julio de 2026

Resumen

Este informe describe el diseño técnico del sistema de gestión de inventario desarrollado para el examen parcial de Aplicaciones Web: una API REST construida con Spring Boot 3.5.16 sobre Java 21, con autenticación y autorización basadas en JSON Web Tokens (JWT), persistencia en PostgreSQL, y una capa de cache con Redis bajo el patrón *cache-aside*. Se documentan la arquitectura por capas, el contrato de respuesta uniforme de la API, el modelo de datos, y las decisiones de seguridad y de caché tomadas durante la implementación.

1. Introducción

El sistema permite gestionar el inventario de productos del Mercado Municipal de Quevedo mediante una API REST protegida por autenticación JWT. Los requisitos funcionales cubiertos son: listado paginado de productos, creación validada, eliminación lógica (*soft delete*), autorización basada en roles (ROLE_USER / ROLE_ADMIN), y una capa de cache con Redis para reducir la carga sobre la base de datos en las operaciones de lectura.

Todo el sistema se distribuye mediante Docker Compose, con tres servicios: la aplicación Spring Boot (app), PostgreSQL 16 (postgres) y Redis 7 (redis).

2. Arquitectura por capas

El proyecto sigue una arquitectura por capas convencional, separando responsabilidades para que cada componente tenga un único motivo de cambio:

- **domain:** entidades JPA (Producto, Usuario) que mapean directamente las tablas de la base de datos.
- **repository:** interfaces JpaRepository (ProductoRepository, UsuarioRepository). Es el *único* punto de acceso a datos: ni los controladores ni los servicios ejecutan consultas directas.
- **service:** lógica de negocio y orquestación de caché (ProductoService). Traduce entidades a DTOs de salida.
- **controller:** adaptadores HTTP delgados (ProductoController, AuthController) que solo mapean peticiones/respuestas y delegan toda la lógica al servicio.
- **dto:** objetos de transferencia (ProductoRequest, ProductoResponse, ApiResponse, etc.), implementados como *records* de Java 21 por su inmutabilidad y menor código repetitivo.
- **security:** todo lo relacionado con JWT y Spring Security (JwtService, JwtAuthenticationFilter, SecurityConfig, manejadores de errores de autenticación).

- **web**: manejo centralizado de excepciones (`GlobalExceptionHandler`) que traduce cualquier excepción de la capa de negocio al contrato de respuesta uniforme de la API.
- **config**: configuración de infraestructura transversal, como la caché de Redis (`CacheConfig`).

Esta separación se aproxima a los principios de diseño de APIs orientadas a recursos descritos por Fielding [1], en cuanto a que cada recurso (`/api/v1/productos`) se manipula mediante los verbos HTTP estándar (GET, POST, DELETE) sin exponer detalles de implementación interna al cliente.

3. Modelo de datos

El esquema (`db/schema.sql`) define dos tablas:

```
CREATE TABLE productos (
  id BIGSERIAL PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  categoria VARCHAR(50) NOT NULL,
  stock INTEGER NOT NULL CHECK (stock >= 0),
  precio DECIMAL(10,2) NOT NULL CHECK (precio >= 0.01),
  activo BOOLEAN NOT NULL DEFAULT TRUE,
  creado_en TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE usuarios (
  id BIGSERIAL PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  password_hash VARCHAR(100) NOT NULL,
  role VARCHAR(20) NOT NULL CHECK (role IN ('ROLE_USER', 'ROLE_ADMIN'))
);
```

Los **CHECK** de `stock` y `precio` garantizan invariantes de negocio a nivel de base de datos (no solo en la capa de validación de la API), y el borrado de productos es siempre lógico: el campo `activo` se pone en `false` en lugar de ejecutar un **DELETE** físico, preservando el historial. El listado solo considera productos con `activo = true` (`ProductoRepository.findByActivoTrue`).

Las contraseñas de `usuarios` se almacenan con **BCrypt** (`password_hash`), siguiendo el esquema adaptativo de Provos y Mazières [6], que incorpora un factor de costo configurable para resistir ataques de fuerza bruta a medida que el hardware mejora.

4. Contrato de respuesta uniforme

Todas las respuestas de la API, exitosas o de error, siguen la misma forma, implementada como el *record* genérico `ApiResponse<T>`:

```
{
  "success": true,
  "data": [ ... ],
  "message": "Listado obtenido correctamente",
  "meta": { "page": 0, "size": 20, "totalElements": 57, "totalPages": 3 }
}
```

En una respuesta de error, el campo `errors` (lista de `{field, message}`) reemplaza a `data/meta`:

```
{
  "success": false,
  "message": "Error de validacion",
  "errors": [ ... ]
}
```

```

"errors": [
  { "field": "nombre", "message": "must not be blank" }
]
}

```

Se optó por una única clase `ApiResponse<T>` (con `meta` y `errors` anotados con `@JsonInclude(NON_NULL)`) en lugar de una jerarquía de tipos de éxito/error separada, de modo que todos los controladores y el manejador global de excepciones devuelvan siempre el mismo tipo, y el cliente de la API pueda inspeccionar únicamente el campo `success` para decidir cómo interpretar la respuesta.

5. Seguridad: autenticación y autorización con JWT

La autenticación es *stateless*: el cliente obtiene un token mediante `POST /api/v1/auth/login` y lo envía en cada petición protegida con el esquema `Authorization: Bearer <token>`, tal como lo define RFC 6750 [3] para el uso de tokens portador (*bearer tokens*).

El token en sí es un JSON Web Token firmado con HMAC-SHA256, según RFC 7519 [2], con dos *claims* relevantes además de los estándar (`sub`, `iat`, `exp`): el nombre de usuario como *subject* y el rol (`ROLE_USER` o `ROLE_ADMIN`) como claim personalizado `role`. El tiempo de expiración se fijó en una hora, un valor razonable para pruebas y demostraciones que limita la ventana de exposición si el token se filtra, sin forzar reautenticaciones constantes.

El flujo de autorización sigue el modelo estándar de Spring Security [7]:

1. `JwtAuthenticationFilter` (un `OncePerRequestFilter`) intercepta cada petición, extrae el token del header `Authorization`, y si es válido puebla el `SecurityContext` con las *authorities* correspondientes.
2. Si no hay token o es inválido, el filtro simplemente deja continuar la cadena sin autenticar; es `AuthorizationFilter` quien más adelante rechaza el acceso a un recurso protegido, delegando el error 401 a un `AuthenticationEntryPoint` personalizado.
3. Un `AccessDeniedHandler` personalizado traduce los rechazos por rol insuficiente (403) al mismo contrato `ApiResponse.error(...)` usado en el resto de la API, en vez del HTML por defecto de Spring Security.

Las reglas de autorización, definidas en `SecurityConfig`, son:

Endpoint	Rol requerido	Sin token / rol insuficiente
<code>POST /api/v1/auth/login</code>	público	—
<code>GET /api/v1/productos</code>	<code>ROLE_USER</code> o <code>ROLE_ADMIN</code>	401
<code>POST /api/v1/productos</code>	<code>ROLE_ADMIN</code>	401 / 403
<code>DELETE /api/v1/productos/{id}</code>	<code>ROLE_ADMIN</code>	401 / 403

Dado que Spring Security no hace que `ROLE_ADMIN` herede automáticamente las autoridades de `ROLE_USER`, el endpoint de listado se declaró explícitamente con `hasAnyRole(“USER”, “.ADMIN”)` en lugar de introducir una jerarquía de roles (`RoleHierarchy`) separada: con un único rol por usuario en el modelo de datos, esta forma es más directa de auditar en un solo lugar.

Como salvaguarda adicional, la construcción de `GrantedAuthority` a partir del valor de rol (ya sea leído de la base de datos o de un claim del JWT) se centralizó en un único método utilitario (`RoleAuthorityUtils`) que normaliza el prefijo `ROLE_` si no está presente, evitando que una futura fuente de datos sin ese prefijo rompa la autorización en silencio.

6. Patrón cache-aside con Redis

El listado paginado de productos (`GET /api/v1/productos`) usa el patrón *cache-aside*, tal como lo describen tanto la documentación de patrones de Azure [5] como Kleppmann [4] en su

discusión sobre sistemas de caché: la aplicación consulta primero la caché: si el valor no está (*cache miss*), calcula el resultado consultando la base de datos, lo guarda en la caché, y lo devuelve; en peticiones subsecuentes con los mismos parámetros de paginación (*cache hit*), el resultado se sirve directamente desde Redis sin volver a consultar PostgreSQL.

En la implementación, esto se logra anotando el método de servicio (no el controlador) con `@Cacheable`:

```
@Cacheable(  
    value = "productos",  
    key = "'page:' + #pageable.pageNumber  
+ #pageable.pageSize  
+ #pageable.sort.toString()"  
)  
public ProductoPageResult listarActivos(Pageable pageable) { ... }
```

La clave de caché incluye página, tamaño y orden para no mezclar resultados de distintas combinaciones de paginación bajo la misma entrada. Cualquier operación de escritura (**crear**, **eliminar**) invalida toda la caché de productos con `@CacheEvict(value = "productos", allEntries = true)`, dado que una única entrada modificada puede afectar el conteo total y la distribución de varias páginas cacheadas simultáneamente.

Para la serialización se usó un `Jackson2JsonRedisSerializer` atado al tipo concreto `ProductoPageResult` (en vez del serializador genérico con tipado polimórfico de Spring Data Redis), ya que los DTOs del proyecto son *records* de Java — por lo tanto, clases **final** — para las que la estrategia de tipado por defecto de Jackson omite la metadata de tipo en el valor raíz, produciendo un documento que el propio Jackson no puede releer. Al conocerse de antemano el único tipo que esta caché almacena, un serializador tipado explícito es más simple y evita ese problema por completo.

7. Manejo centralizado de errores

`GlobalExceptionHandler (@RestControllerAdvice)` concentra la traducción de excepciones al contrato `ApiResponse.error(...)`:

- `AuthenticationException` (credenciales inválidas en el login) → 401.
- `MethodArgumentNotValidException` (fallos de validación de `@Valid`) → 400, con la lista de errores por campo.
- `ProductoNoEncontradoException` (id inexistente en DELETE) → 404.

Centralizar este mapeo en un único componente evita duplicar la construcción de la respuesta de error en cada controlador, y mantiene el formato de salida consistente en toda la API.

8. Conclusiones

El sistema implementado cubre los cinco requisitos funcionales del examen (listado paginado, creación validada, eliminación lógica, autorización basada en roles, y caché con Redis) sobre una arquitectura por capas convencional que aísla claramente el acceso a datos, la lógica de negocio y la capa HTTP. Las decisiones de seguridad (JWT stateless, BCrypt, normalización centralizada de roles) y de caché (cache-aside con invalidación total en escritura, y serialización tipada para evitar los problemas de tipado polimórfico de Jackson con *records*) se documentaron junto con su justificación técnica, de forma que puedan auditarse y, si corresponde, revisarse en iteraciones futuras del proyecto.

Referencias

- [1] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [2] Michael B. Jones, John Bradley, and Nat Sakimura. JSON Web Token (JWT). Technical Report RFC 7519, IETF, May 2015.
- [3] Michael B. Jones and Dick Hardt. The OAuth 2.0 Authorization Framework: Bearer Token Usage. Technical Report RFC 6750, IETF, October 2012.
- [4] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, Sebastopol, CA, 2017.
- [5] Microsoft. Cache-Aside pattern – Azure Architecture Center, 2023.
- [6] Niels Provos and David Mazières. A future-adaptable password scheme. In *Proceedings of the 1999 USENIX Annual Technical Conference*, Monterey, CA, 1999. USENIX Association.
- [7] VMware. Spring Security Reference Documentation, 2024.